TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 9

Health, Autoscaling and Troubleshooting

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 9: Health checks, autoscaling, monitoring and troubleshooting 3

9.1 Health probes 3

9.2 Autoscaling 5

9.3 Observability 6

9.4 Troubleshooting playbook 6

9.5 Live lab 7

Class 9 - Top 20 interview questions 8

K8S · CLASS 09 · 60-90 minutes

Health checks, autoscaling, monitoring and troubleshooting

By the end you can make apps self-heal with probes, scale automatically on load, observe the cluster, and debug the most common failures quickly.

What you will learn today

- Liveness, readiness and startup probes - HTTP, TCP, exec, gRPC
- Graceful shutdown: preStop hooks and termination grace period
- HorizontalPodAutoscaler with metrics-server; VPA and Cluster Autoscaler
- PodDisruptionBudget for safe maintenance
- Monitoring and logging: kubectl top, Prometheus, Grafana, Loki
- A troubleshooting playbook for the common errors
- Lab: probe failures and a live autoscaling demo

Part	Topic	Time
1	Story and probes	15 min
2	Autoscaling and PDB	15 min
3	Observability and troubleshooting	15 min
4	Live lab	25 min
5	Recap and interview questions	10 min

9.1 Health probes

Ravi's story

During a sale the payment service froze - the process was alive but answered nothing. Kubernetes saw a running container and kept sending it traffic. Another day, new pods received orders before they had connected to the database, and the first 200 customers got errors. Ravi needed Kubernetes to know not just "is it running?" but "is it healthy?" and "is it ready?"

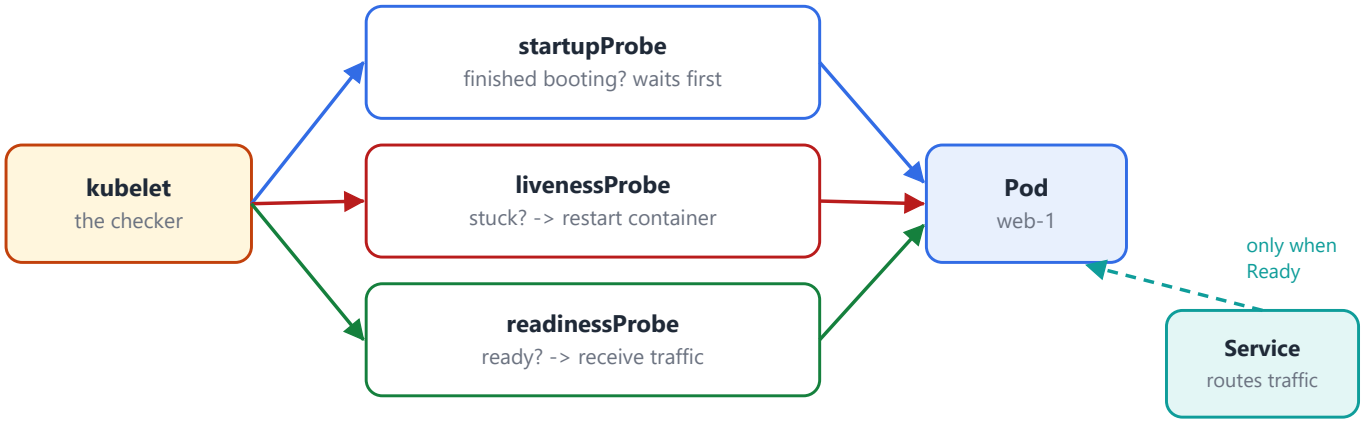


Figure 9.1 - The kubelet runs three kinds of probes; only Ready pods get Service traffic

Probe	Question	On failure
startupProbe	Has the app finished starting?	Keep waiting; other probes are paused. Kill after failureThreshold
livenessProbe	Is the app still alive or stuck?	Container is restarted
readinessProbe	Can it serve traffic now?	Pod removed from Service endpoints (not restarted)

Everyday example

Probes are like a hospital. The **startup** check: is the patient awake after surgery? Do not judge anything else until then. The **liveness** check: is the heart beating? If not, resuscitate (restart). The **readiness** check: is the doctor ready to see patients? If on a break, the receptionist simply stops sending patients for a while.

Probe configuration

```
containers:
  - name: api
    image: my-api:2.1
    ports: [{ containerPort: 8080 }]
    startupProbe:
      httpGet: { path: /healthz, port: 8080 }
      failureThreshold: 30          # 30 x 5s = up to 150s to start
      periodSeconds: 5
    livenessProbe:
      httpGet: { path: /healthz, port: 8080 }
      periodSeconds: 10
      failureThreshold: 3
    readinessProbe:
      httpGet: { path: /ready, port: 8080 }   # checks DB connection, etc.
      periodSeconds: 5
    lifecycle:
      preStop:
        exec: { command: ["sh", "-c", "sleep 10"] }   # let load balancers drain
terminationGracePeriodSeconds: 30
```

Probe mechanisms: `httpGet` (2xx/3xx = healthy), `tcpSocket` (port opens), `exec` (command exits 0) and `grpc`. Useful tuning fields: `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `failureThreshold`, `successThreshold`.

When a pod is deleted, Kubernetes removes it from endpoints, runs the **preStop** hook, sends **SIGTERM**, waits up to `terminationGracePeriodSeconds` (default 30) and finally sends **SIGKILL**. Apps should finish in-flight requests on SIGTERM.

Common mistake

- Making the liveness probe check the database. If the DB blips, every pod restarts at once. Liveness should only check the process itself; put dependencies in readiness.
- No startupProbe for slow apps (Spring Boot, large Java apps) - liveness kills them before they finish booting.

9.2 Autoscaling

Autoscaler	Scales	Based on
HorizontalPodAutoscaler (HPA)	Number of pod replicas	CPU, memory, or custom/external metrics
VerticalPodAutoscaler (VPA)	Requests/limits of each pod	Historical usage (add-on)
Cluster Autoscaler / Karpenter	Number of nodes	Pending pods that do not fit / idle nodes
KEDA	Replicas, even to zero	Events: queue length, Kafka lag, cron

Everyday example
A supermarket opens extra billing counters when queues grow and closes them when it is quiet - that is the HPA. Giving one cashier a faster scanner is the VPA. Renting the shop next door when there is no space for more counters is the Cluster Autoscaler.

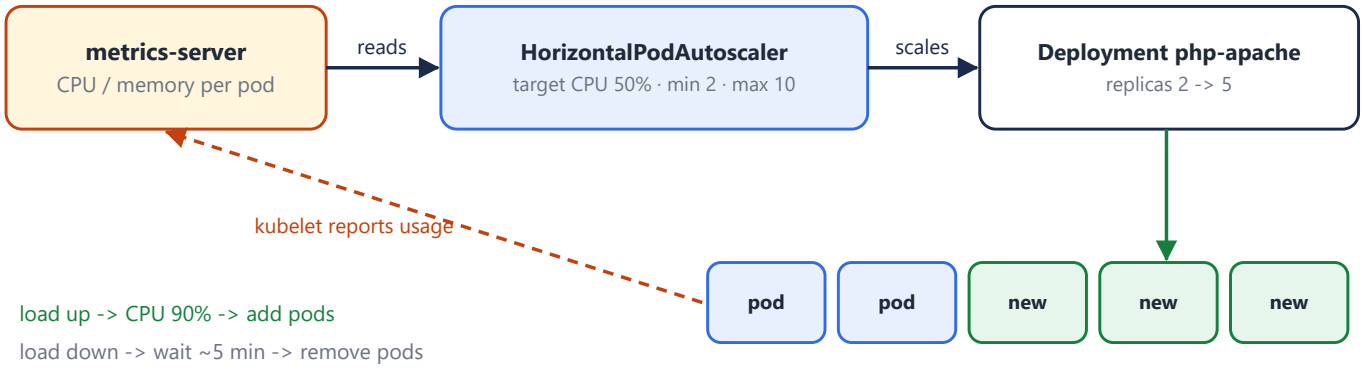


Figure 9.2 - HPA reads metrics and changes the Deployment's replica count

The HPA formula: `desired = ceil(current replicas x current metric / target)`. With 2 pods at 90% CPU and a 50% target: ceil(2 x 90 / 50) = 4 pods. Scale-down waits for a stabilisation window (5 minutes by default) to avoid flapping. HPA needs **requests** to be set, because CPU percentage is measured against the request.

hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 50 }
```

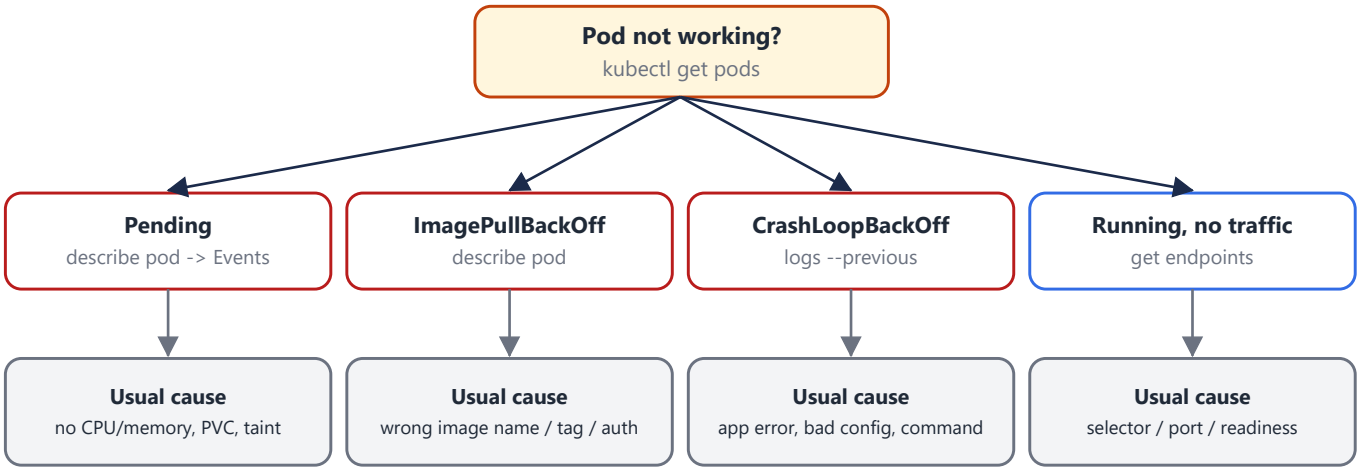
PodDisruptionBudget

A **PDB** limits how many pods of an app may be down at once during *voluntary* disruptions such as `kubectl drain` or cluster upgrades: `minAvailable: 2` or `maxUnavailable: 1`. It does not protect against node crashes.

9.3 Observability

Need	Tool
Quick CPU/memory numbers	`kubectl top nodes`, `kubectl top pods` (needs metrics-server)
Metrics, alerts, dashboards	Prometheus + Alertmanager + Grafana (kube-prometheus-stack Helm chart)
Central logs	Fluent Bit / Fluentd shipping to Loki, Elasticsearch/OpenSearch or a cloud service
Distributed tracing	OpenTelemetry with Jaeger or Tempo
Cluster events	`kubectl get events -A --sort-by=.lastTimestamp`

9.4 Troubleshooting playbook



Golden order: get -> describe -> logs -> events -> exec

Figure 9.3 - Start from the pod status and follow the matching path

Symptom	First command	Typical fix
Pending	`kubectl describe pod`	Lower requests, add nodes, fix nodeSelector/taint, bind the PVC
ErrImagePull / ImagePullBackOff	`kubectl describe pod`	Correct image/tag, add imagePullSecrets
CrashLoopBackOff	`kubectl logs --previous`	Fix the app error, command, env or config
OOMKilled (exit 137)	`kubectl describe pod`	Raise memory limit or fix the memory leak
CreateContainerConfigError	`kubectl describe pod`	Create the missing ConfigMap/Secret or key
Running but 0/1 Ready	`kubectl describe pod`	Fix the readiness probe path/port or the dependency
Service not reachable	`kubectl get endpointslices`	Fix selector labels, targetPort, NetworkPolicy
Node NotReady	`kubectl describe node`	Check kubelet, disk/memory pressure, network

For images without a shell, attach a debugging container: `kubectl debug -it <pod> --image=busybox:1.36 --target=<container>`. `kubectl debug node/<node> -it --image=busybox:1.36` opens a shell on a node.

9.5 Live lab

liveness.yaml - after ~45 s RESTARTS goes up

```
apiVersion: v1
kind: Pod
metadata:
  name: liveness-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      args: ["sh", "-c", "touch /tmp/healthy; sleep 30; rm -f /tmp/healthy; sleep 600"]
      livenessProbe:
        exec: { command: ["cat", "/tmp/healthy"] }
        initialDelaySeconds: 5
        periodSeconds: 5
```

PowerShell

```
kubectl apply -f liveness.yaml
kubectl get pod liveness-demo -w
kubectl describe pod liveness-demo        # "Liveness probe failed: cat: can't open..."

minikube addons enable metrics-server
kubectl create deployment php-apache --image=registry.k8s.io/hpa-example
kubectl set resources deployment php-apache --requests=cpu=200m --limits=cpu=500m
kubectl expose deployment php-apache --port=80
kubectl apply -f hpa.yaml
kubectl get hpa -w                         # keep this window open

# second window - generate load:
kubectl run load --rm -it --image=busybox:1.36 -- `
  sh -c "while true; do wget -q -O- http://php-apache; done"
# watch TARGETS climb above 50% and REPLICAS grow; press Ctrl+C to stop load
kubectl top pods
```

After you stop the load, replicas return to 1 after about five minutes. Clean up: `kubectl delete hpa php-apache; kubectl delete deploy,svc php-apache; kubectl delete pod liveness-demo`.

Class 9 recap

- Startup protects slow boots; liveness restarts; readiness removes from traffic.
- Graceful shutdown: preStop -> SIGTERM -> grace period -> SIGKILL.
- HPA scales pods (needs requests + metrics-server); VPA resizes; Cluster Autoscaler adds nodes.
- PDB keeps a minimum running during drains and upgrades.
- Debug order: get -> describe -> logs -> events -> exec/debug.

Class 9 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What are liveness, readiness and startup probes?

Liveness checks whether a container should be restarted; readiness checks whether it should receive traffic; startup checks whether the application has finished starting and holds off the other probes until it has.

Q2. What happens when a liveness probe fails versus a readiness probe?

A failing liveness probe makes the kubelet restart the container. A failing readiness probe removes the pod from Service endpoints without restarting it, and it is added back when it passes again.

Q3. When do you need a startup probe?

For applications with long or unpredictable start-up times, such as large Java apps. It prevents the liveness probe from killing the container before it is up.

Q4. What probe mechanisms are available?

httpGet (success on status 200-399), tcpSocket (port accepts a connection), exec (command exits 0) and grpc.

Q5. Name important probe parameters.

initialDelaySeconds, periodSeconds, timeoutSeconds, failureThreshold and successThreshold.

Q6. Should a liveness probe check the database?

No. If the database has a problem every pod would restart, making things worse. Liveness should check only the process; dependency checks belong in readiness.

Q7. Explain graceful termination of a pod.

The pod is marked Terminating and removed from endpoints, the preStop hook runs, the container gets SIGTERM, Kubernetes waits up to terminationGracePeriodSeconds (default 30) and then sends SIGKILL.

Q8. What is the HorizontalPodAutoscaler?

A controller that adjusts the replica count of a Deployment, StatefulSet or ReplicaSet based on observed metrics such as CPU, memory, or custom and external metrics, within min and max bounds.

Q9. What are the prerequisites for CPU-based HPA?

A metrics source such as metrics-server and CPU requests defined on the containers, because utilisation is calculated as a percentage of the request.

Q10. How does the HPA calculate the desired replicas?

desiredReplicas = ceil(currentReplicas x currentMetricValue / targetValue). Scale-down uses a stabilisation window (300 seconds by default) to avoid flapping.

Q11. What is the VerticalPodAutoscaler?

An add-on that recommends or automatically sets CPU and memory requests based on historical usage. It should not be combined with HPA on the same CPU or memory metric.

Q12. What is the Cluster Autoscaler?

A component that adds nodes when pods are Pending because of insufficient resources and removes under-used nodes. Karpenter is a faster, more flexible alternative on AWS and other clouds.

Q13. What is a PodDisruptionBudget?

A policy limiting how many pods of an application can be unavailable during voluntary disruptions like drains and upgrades, using minAvailable or maxUnavailable.

Q14. How do you check resource usage of pods and nodes?

`kubectl top pods` and `kubectl top nodes`, which need metrics-server. For history and alerts use Prometheus and Grafana.

Q15. How is logging typically done in Kubernetes?

Apps write to stdout/stderr; the runtime stores logs on the node; a DaemonSet agent such as Fluent Bit ships them to a central store such as Loki, Elasticsearch or a cloud logging service.

Q16. How would you troubleshoot a pod in CrashLoopBackOff?

Check `kubectl logs --previous`, then `kubectl describe pod` for exit code, reason (OOMKilled) and events. Verify command, args, env vars, ConfigMaps/Secrets, probes and resource limits; reproduce with `kubectl debug` if needed.

Q17. What does exit code 137 mean?

The container was killed with SIGKILL (128 + 9), usually OOMKilled for exceeding its memory limit, or killed after the grace period.

Q18. How do you debug a container that has no shell?

Use `kubectl debug -it <pod> --image=busybox --target=<container>` to attach an ephemeral container that shares the process namespace, or copy the pod with `--copy-to`.

Q19. How do you view cluster events?

`kubectl get events -n <ns> --sort-by=.lastTimestamp` or the Events section of `kubectl describe`. Events are kept for about one hour by default.

Q20. What is the difference between monitoring and observability?

Monitoring watches known metrics and alerts on thresholds. Observability combines metrics, logs and traces so you can ask new questions and understand unknown failures.